

Exercise 2: E-commerce Platform Search Function

Data Structures and Algorithms | Cognizant Deep Skilling 5.0 | Sanskar Kumawat

Step 1: Understanding Asymptotic Notation

What is Big O Notation?

Big O notation is a mathematical way to describe how the runtime of an algorithm grows as the size of the input (n) increases. It helps us compare algorithms without worrying about hardware speed or exact execution time — we only care about the growth pattern.

Think of it like this: if you have 1000 products vs 1 million products, Big O tells you how much slower the algorithm gets. This is crucial for e-commerce platforms where product catalogues can grow from hundreds to millions.

Common Big O Complexities (best to worst):

$O(1)$	→ Constant time	— always same speed, no matter how big the input
$O(\log n)$	→ Logarithmic	— halves the problem each step (Binary Search)
$O(n)$	→ Linear	— grows directly with input size (Linear Search)
$O(n \log n)$	→ Linearithmic	— typical of efficient sorting algorithms
$O(n^2)$	→ Quadratic	— nested loops, gets slow very fast

Best, Average, and Worst Case Scenarios

Every search algorithm has three scenarios based on where the target element is:

- **Best Case (O notation: Ω Omega)** - The element is found immediately (e.g., it is the very first item checked). Algorithm exits early with minimum work.
- **Average Case (O notation: Θ Theta)** - The element is somewhere in the middle of the dataset. This is the realistic everyday scenario for most searches.
- **Worst Case (O notation: O Big-O)** - The element is at the very end OR does not exist at all. The algorithm does maximum work. This is what Big O typically refers to.

Real-world example:

Searching for productId 999 in a list of 1000 products:

- Linear Search — checks all 1000 items before finding it (worst case $O(n)$)
- Binary Search — finds it in ~10 comparisons ($\log_2 1000 \approx 10$, worst case $O(\log n)$)

Step 2: Setup: Product Class

The Product class models an item in the e-commerce catalogue. It has three attributes used for searching: productId (used as the search key), productName, and category.

```
// Product.java

public class Product {
    int productId;
    String productName;
    String category;

    public Product(int productId, String productName, String category) {
        this.productId = productId;
        this.productName = productName;
        this.category = category;
    }
    @Override
    public String toString() {
        return "Product{ID=" + productId + ", Name='" + productName + "',
Category='" + category + "'}";
    }
}
```

Step 3: Implementation

Linear Search

Linear Search goes through every element one by one from the start and checks if the productId matches the target. It requires NO sorting and works on any array as-is.

Time Complexity:

- Best case $O(1)$: product is the very first element
- Average case $O(n)$: product is somewhere in the middle
- Worst case $O(n)$: product is last OR does not exist at all

```
// Linear Search - searches an UNSORTED array by productId
// No sorting required. Checks each element one by one.
```

```

public static int linearSearch(Product[] products, int targetId) {
    for (int i = 0; i < products.length; i++) {
        if (products[i].productId == targetId) {
            return i;
        }
    }
    return -1;
}

```

Binary Search

Binary Search divides the sorted array in half each time. It checks the middle element, and based on whether the target is smaller or larger, it discards one entire half and repeats on the remaining half.

Time Complexity:

- Best case $O(1)$: product is exactly at the middle on the first check
- Average case $O(\log n)$: cuts the search space in half each step
- Worst case $O(\log n)$: keeps halving until only one element remains

IMPORTANT: The array MUST be sorted by productId before calling binary search.

```

// Binary Search - searches a SORTED array by productId
// REQUIREMENT: products array must be sorted by productId ascending.

public static int binarySearch(Product[] products, int targetId) {
    int left = 0;
    int right = products.length - 1;

    while (left <= right) {
        int mid = left + (right - left) / 2;

        if (products[mid].productId == targetId) {
            return mid;
        } else if (products[mid].productId < targetId) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    return -1;
}

```

Test Class: Main.java

The test class demonstrates both searches running on the same product catalogue and prints results including a case where the product does not exist.

```
public class Main {
    public static void main(String[] args) {

        Product[] linearProducts = {
            new Product(105, "Headphones", "Electronics"),
            new Product(101, "Laptop", "Electronics"),
            new Product(110, "Watch", "Accessories"),
            new Product(103, "Book", "Education"),
            new Product(108, "Shoes", "Fashion")
        };

        Product[] binaryProducts = {
            new Product(101, "Laptop", "Electronics"),
            new Product(103, "Book", "Education"),
            new Product(105, "Headphones", "Electronics"),
            new Product(108, "Shoes", "Fashion"),
            new Product(110, "Watch", "Accessories")
        };

        int targetId = 108;
        int linearResult = SearchAlgorithm.linearSearch(linearProducts,
targetId);

        if (linearResult != -1) {
            System.out.println("Linear Search Result:");
            System.out.println(linearProducts[linearResult]);
        } else {
            System.out.println("Product not found using Linear Search.");
        }
        System.out.println();

        int binaryResult = SearchAlgorithm.binarySearch(binaryProducts,
targetId);

        if (binaryResult != -1) {
            System.out.println("Binary Search Result:");
            System.out.println(binaryProducts[binaryResult]);
        } else {
            System.out.println("Product not found using Binary Search.");
        }
    }
}
```

Step 4: Analysis

Time Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

Detailed Comparison

Linear Search Pros and Cons:

- Does NOT require the array to be sorted works on any dataset as-is
- Simple to implement and easy to understand
- Suitable for small datasets where sorting overhead is not worth it
- Becomes very slow as dataset grows $O(n)$ means 1 million products = up to 1 million checks

Binary Search Pros and Cons:

- Extremely fast - $O(\log n)$ means 1 million products = only ~20 comparisons
- Scales very well as the product catalogue grows
- Requires the array to be sorted by productId before searching
- Sorting has its own cost - $O(n \log n)$, but this is a one-time setup cost

Which Algorithm is More Suitable for an E-commerce Platform?

Conclusion: Binary Search is significantly more suitable for a real e-commerce platform.

Reasons:

1. Scale : E-commerce platforms like Amazon or Flipkart have millions of products. Binary Search handles this with $O(\log n)$ efficiency while Linear Search would be impractically slow.
2. Search frequency : Users search hundreds of times per second. Every millisecond saved in search time multiplies across all requests.
3. Sorted catalogue : Product databases are maintained in sorted order by ID anyway. The sorting cost is a one-time setup, not a per-search cost.
4. Practical example : For 1,000,000 products: Linear Search checks up to 1,000,000 elements. Binary Search checks at most $\log_2(1,000,000) \approx 20$ elements. That is a 50,000x improvement in the worst case.

Expected Output

```
Linear Search Result:
```

```
Product{ID=108, Name='Shoes', Category='Fashion'}
```

```
Binary Search Result:
```

```
Product{ID=108, Name='Shoes', Category='Fashion'}
```